MARS Package Documentation

Multivariate Adaptive Regression Splines

STAT 360 - Advanced R for Data Science

Final Project

GitHub Repository:

https://github.com/Jassa47/STAT360-MARS-Project

Authors: Akaaljot Mathoda (301575954), Mayank Mayank (301563549)

Contents

I mars() Function Documentation **4**

1 Description **4**

2 Usage **4**

3 Arguments **4**

4 Details **5**
- 4.1 Forward Stepwise Pass (Algorithm 2) . . . 5
- 4.2 Lack-of-Fit Criterion (GCV) . . . 5
- 4.3 Backward Pruning Pass (Algorithm 3) . . . 6
- 4.4 Model Fitting . . . 6
- 4.5 Data Structures . . . 6

5 Value **6**

6 Author(s) **7**

7 References **7**

8 See Also **8**

9 Examples **8**
- 9.1 Example 1: Course-Provided Test Data (marstestdata) . . . 8
- 9.2 Example 2: Boston Housing Data . . . 9
- 9.3 Example 3: Simulated Nonlinear Data with Train/Test Split . . . 9

II How fwd_stepwise() Works **11**

1 Overview **11**

2 Key R Variables **11**

3 Step-by-Step Algorithm Walkthrough **12**
- 3.1 Step 1: Initialization . . . 12
- 3.2 Step 2: Outer Loop — Iterating Over Pairs . . . 12
- 3.3 Step 3: Inner Loops — Searching for the Best Split . . . 12
- 3.4 Step 4: Updating B and Bfuncs . . . 13

4 Worked Example: Step-by-Step Trace **13**
- 4.0.1 Iteration $i = 1$ ($M = 1$) . . . 13
- 4.0.2 Result . . . 14

5 The Hinge Function h() **14**

6 Support Functions **14**
6.1 init_B(N, Mmax) 14
6.2 split_points(xv, Bm) 14
6.3 LOF(form, data, control) 15

III Backward Elimination Algorithm **16**

1 Overview **16**

2 Notation and Symbols **16**

3 Algorithm Description **16**
3.1 Step 1: Initialization 16
3.2 Step 2: Iterative Removal 17
3.3 Step 3: Final Model 17

4 Why Backward Pruning Is Necessary **17**

5 Illustrative Example **18**

6 Comparison to Forward Pass Only **18**

Part I

mars() Function Documentation

1. Description

Fit a Multivariate Adaptive Regression Splines (MARS) model to data. The `mars()` function builds an additive model of piecewise-linear basis functions (hinge functions and their products) using the two-phase algorithm of Friedman (1991): a forward stepwise pass that greedily adds basis-function pairs to reduce a lack-of-fit criterion, followed by a backward pruning pass that removes redundant terms using generalized cross-validation (GCV).

2. Usage

```
mars(formula, data, control = mars.control())

mars.control(Mmax = 2, d = 3, trace = FALSE)
```

3. Arguments

`formula`
: An object of class `formula` specifying the response and predictor variables, following the conventions of `lm()`. For example, `y ~ .` fits the response `y` against all remaining columns in `data`.

`data`
: A data frame containing the variables referenced in `formula`. Both the response and all predictors must be present as columns. Missing values are not currently handled and should be removed prior to fitting.

`control`
: A `mars.control` object (or a list coercible to one) specifying algorithm tuning parameters. Created by the helper function `mars.control()`, which accepts the following sub-arguments:

  `Mmax`
  : A positive even integer giving the maximum number of non-constant basis functions to build in the forward pass. Must be at least 2; odd values are rounded up to the next even integer. Default is `2L`.

`d` A numeric smoothing parameter used in the GCV penalty. Controls the cost per basis function: larger values of `d` produce sparser models. Default is `3`, following Friedman's recommendation.

`trace`
Logical. If `TRUE`, diagnostic information (current basis index, candidate splits, LOF values) is printed during both the forward and backward passes. Default is `FALSE`.

4. Details

The MARS algorithm, introduced by Friedman (1991), is a flexible non-parametric regression technique that automatically discovers non-linearities and interactions in the data. It extends linear models by constructing basis functions from piecewise-linear *hinge functions* of the form $h(x-t) = \max(0, x-t)$ and $h(t-x) = \max(0, t-x)$, where t is a knot location chosen from observed data values. Products of such hinge functions capture interaction effects between variables.

4.1 Forward Stepwise Pass (Algorithm 2)

The forward pass begins with a model containing only the constant (intercept) basis function $B_0(\mathbf{x}) = 1$. At each step, the algorithm searches over all existing basis functions B_m, all eligible predictor variables x_v, and all candidate knot locations t to find the split that most reduces the lack-of-fit (LOF) criterion. The selected parent basis function B_m is multiplied by a pair of reflected hinge functions to produce two new basis functions:

$$B_M(\mathbf{x}) = B_{m^*}(\mathbf{x}) \cdot [-(x_{v^*} - t^*)]_+ \tag{1}$$
$$B_{M+1}(\mathbf{x}) = B_{m^*}(\mathbf{x}) \cdot [+(x_{v^*} - t^*)]_+ \tag{2}$$

Here m^*, v^*, and t^* denote the optimal parent, variable, and knot, respectively. Crucially, the parent basis function is **retained** in the model (unlike recursive partitioning), and two new children are **appended**. This continues for `Mmax/2` iterations, adding pairs of basis functions until up to `Mmax` non-constant basis functions have been created.

To avoid redundant models, only predictor variables not already involved in basis function B_m are considered for splitting (variable eligibility constraint). Candidate split points for variable x_v are the sorted unique values of x_v in the region where $B_m(\mathbf{x}) > 0$, excluding the maximum.

4.2 Lack-of-Fit Criterion (GCV)

Model quality is measured by the Generalized Cross-Validation (GCV) criterion, which approximates leave-one-out cross-validation without the computational expense of refitting:

$$\mathrm{GCV}(M) = \frac{N \cdot \mathrm{RSS}(M)}{\left(N - \widetilde{C}(M)\right)^2} \tag{3}$$

where $\mathrm{RSS}(M)$ is the residual sum of squares of the current model, N is the sample size, and $\widetilde{C}(M)$ is the effective number of parameters. Friedman defines $\widetilde{C}(M) = \mathrm{trace}(\mathbf{H}) + d \cdot M$, where $\mathbf{H}$ is

the hat matrix and d is the smoothing parameter from `mars.control()`. In the implementation, $\widetilde{C}(M)$ is computed as `sum(hatvalues(ff)) + control$d * M`, with M derived from the number of fitted coefficients minus one.

4.3 Backward Pruning Pass (Algorithm 3)

The forward pass deliberately overfits to capture all potentially useful basis functions. The backward pass then prunes the model by iteratively removing the term whose deletion causes the smallest increase in GCV. Starting from the full set of $M_{\max} + 1$ basis functions (including the intercept), at each step the algorithm evaluates every subset obtained by removing one non-intercept basis function and selects the subset with the lowest GCV. The intercept term B_0 is never removed. The final model is the subset (across all sizes considered) that achieves the overall minimum GCV. The backward pass uses the formula `y ~ . - 1` to avoid double-counting the intercept, since `B0` already serves as a column of ones.

4.4 Model Fitting

Once the optimal set of basis functions has been determined by the backward pass, the final model coefficients are estimated by ordinary least squares via:

```
lm(y ~ . - 1, data = data.frame(y = y, bwd$B))
```

The returned `mars` object inherits from class `lm`, so standard methods like `residuals()`, `fitted()`, and `coef()` work as expected.

4.5 Data Structures

Internally, the algorithm maintains two key data structures. `B` is an $N \times (M_{\max} + 1)$ data frame where each column stores the evaluated values of a basis function across all observations, initialized by `init_B()` with the first column (`B0`) set to all ones and the remaining columns set to zero. `Bfuncs` is a list of length $M_{\max} + 1$, where each element is either `NULL` (for the intercept) or a matrix with columns (s, v, t) recording the sign, variable index, and knot for each hinge function composing that basis function. The m-th basis function is the product of hinge functions:

$$B_m(\mathbf{x}) = \prod_{k=1}^{K_m} h\big(s_{km} \cdot (x_{v(k,m)} - t_{km})\big)$$

5. Value

`mars()` returns an S3 object of class `c("mars", "lm")`. Since it inherits from `lm`, all standard linear-model extraction functions (e.g., `coef()`, `residuals()`, `fitted()`) are available. In addition to the components inherited from `lm`, the object contains the following MARS-specific components:

`call` The matched call to `mars()`, captured via `match.call()`.

`formula`
: The original formula passed to `mars()`.

`y` The response vector extracted from the model frame.

`B` A data frame of evaluated basis functions, with columns named `B0, B1, ...`. After the backward pass, only the columns corresponding to the selected basis functions are retained.

`Bfuncs` A list of basis-function definitions. Each element is either `NULL` (for the intercept B_0) or a matrix with columns `s` (sign: $+1$ or -1), `v` (variable index), and `t` (knot value). Only the entries corresponding to selected basis functions are present after pruning.

`x_names`
: A character vector of predictor variable names, derived from the model matrix (with the intercept column removed).

`coefficients`
: Named vector of estimated coefficients for the selected basis functions.

`residuals`
: Residuals from the final least-squares fit.

`fitted.values`
: Fitted values from the final model.

`effects, rank, qr, ...`
: Additional components inherited from the internal `lm()` fit.

`mars.control()` returns a named list of class `"mars.control"` with components `Mmax`, `d`, and `trace`.

6. Author(s)

Akaaljot Mathoda () and Mayank Mayank (301563549), Simon Fraser University, STAT 360 - Advanced R for Data Science.

7. References

Friedman, J. H. (1991). "Multivariate Adaptive Regression Splines." *The Annals of Statistics*, 19(1), 1–141. DOI: 10.1214/aos/1176347963.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd Edition. Springer. Chapter 9: Additive Models, Trees, and Related Methods.

Wickham, H. (2019). *Advanced R*, 2nd Edition. CRC Press. (Course textbook for STAT 360.)

R Core Team (2023). *Writing R Extensions*. https://cran.r-project.org/doc/manuals/r-release/R-exts.html.

8. See Also

The following S3 methods are implemented for `mars` objects, providing a user interface consistent with `lm()` and `glm()`:

`print.mars()`
: Prints the model call and estimated coefficients for the selected basis functions, mirroring the output style of `print.lm()`.

`summary.mars()`
: Produces a comprehensive summary including: a human-readable listing of basis functions and their component hinge functions (with variable names, signs, and knot values), followed by the full output of `summary.lm()` — the original call, residual quantiles, a coefficient table with standard errors, t-values, and p-values, plus R-squared and F-statistic.

`predict.mars()`
: Generates predictions for new data. If `newdata` is omitted, returns fitted values from the training data. When `newdata` is supplied, the relevant predictor columns are extracted using the stored `x_names`, and basis functions are reconstructed via the internal helper `make_B()` using the stored `Bfuncs` definitions. Predictions are computed as $\hat{y} = B\hat{\beta}$.

`plot.mars()`
: Produces diagnostic plots of each non-intercept basis function. For basis functions involving exactly one variable, a 2D line plot is generated showing the basis function's shape across the range of its knot values. For basis functions involving exactly two variables, a 3D perspective plot is generated using `graphics::persp()`.

`anova.mars()`
: Performs a sequential analysis of variance (Type I ANOVA) on the fitted MARS model by delegating to the underlying `anova.lm()` method via `NextMethod()`, decomposing the total sum of squares by basis function.

In addition, `residuals()` and `fitted()` are inherited directly from the `lm` class and work without modification. The helper `mars.control()` creates control objects, and the internal function `h()` implements the hinge function.

9. Examples

The following three examples correspond to the test suite in `test.R` and illustrate all functions and methods.

9.1 Example 1: Course-Provided Test Data (marstestdata)

This example uses the `marstestdata` dataset bundled with the package, which contains a response variable `y` and predictors `x1, ..., x5` with 100 observations.

```
library(mars)

data("marstestdata", package = "mars")
```

```
mc  <- mars.control(Mmax = 10, d = 3, trace = FALSE)
fit1 <- mars(y ~ ., data = marstestdata, control = mc)

print(fit1)
summary(fit1)
plot(fit1)
head(predict(fit1))
predict(fit1, newdata = head(marstestdata, 5))
anova(fit1)
```

9.2 Example 2: Boston Housing Data

This example demonstrates MARS on a classic real-world dataset. Median home values in Boston suburbs depend non-linearly on crime rate, number of rooms, tax rate, and other socioeconomic variables. MARS with `Mmax = 20` is allowed to build a rich set of basis functions before backward pruning selects the most predictive subset.

```
library(MASS)
data("Boston")

mc2  <- mars.control(Mmax = 20, d = 3, trace = FALSE)
fit2 <- mars(medv ~ ., data = Boston, control = mc2)

print(fit2)
summary(fit2)
plot(fit2)

# Predictions on first 5 rows
preds2 <- predict(fit2, newdata = head(Boston, 5))
print(data.frame(actual    = head(Boston$medv, 5),
                 predicted = round(preds2, 3)))

# In-sample R-squared
ss_res <- sum((Boston$medv - predict(fit2))^2)
ss_tot <- sum((Boston$medv - mean(Boston$medv))^2)
cat(sprintf("In-sample R-squared: %.4f\n", 1 - ss_res/ss_tot))

anova(fit2)
```

9.3 Example 3: Simulated Nonlinear Data with Train/Test Split

This example applies MARS to a synthetic dataset with known non-linear structure, including `sin()`, quadratic, and absolute value terms. A train/test split evaluates out-of-sample predictive accuracy.

```
set.seed(42)
n  <- 200
x1 <- runif(n, -2, 2)
x2 <- runif(n, -2, 2)
x3 <- runif(n, -2, 2)
y  <- sin(pi*x1) + (x2 - 0.5)^2 + abs(x3) + rnorm(n, sd = 0.3)
```

```
dat3 <- data.frame(y, x1, x2, x3)

# Train/test split
set.seed(7)
idx   <- sample(n, 160)
train <- dat3[idx, ]
test  <- dat3[-idx, ]

mc3  <- mars.control(Mmax = 12, d = 3, trace = FALSE)
fit3 <- mars(y ~ x1 + x2 + x3, data = train, control = mc3)

print(fit3)
summary(fit3)
plot(fit3)

preds3 <- predict(fit3, newdata = test)
rmse   <- sqrt(mean((test$y - preds3)^2))
cat(sprintf("Test RMSE: %.4f\n", rmse))

anova(fit3)
```

Part II

How fwd_stepwise() Works

1. Overview

The function `fwd_stepwise(y, x, control)` implements Algorithm 2 (MARS — Forward Stepwise) from Friedman (1991). It takes the response vector `y`, the predictor matrix `x` (with the intercept column already removed by the caller `mars()`), and a `mars.control` object. It returns a list with components `y`, `B` (a data frame), and `Bfuncs` (a list).

2. Key R Variables

Variable	Type	Description
`N`	integer	Sample size, equal to `length(y)`.
`n`	integer	Number of predictor variables, equal to `ncol(x)`.
`B`	data.frame	$N \times (M_{\max} + 1)$ data frame of evaluated basis functions. Column 1 (`B0`) is initialized to all 1s (the intercept); remaining columns are initialized to 0 and filled as new basis functions are created.
`Bfuncs`	list	List of length $M_{\max} + 1$. Element 1 is `NULL` (intercept has no hinge components). Each subsequent element becomes a matrix with columns (s, v, t) recording the hinge functions that define that basis function.
`lof_best`	numeric	Tracks the best (lowest) GCV value found during the search over all (m, v, t) candidates within one iteration. Reset to `Inf` at the start of each outer-loop iteration.
`split_best`	named vector	Stores the optimal (m, v, t) triple for the current iteration. Updated whenever a candidate split achieves a lower GCV than `lof_best`.
`svars`	integer vector	Indices of predictor variables eligible for splitting basis function B_m. If `Bfuncs[[m]]` is `NULL` (the intercept), `svars = 1:n`. Otherwise, variables whose indices appear in the `v` column of `Bfuncs[[m]]` are excluded via `setdiff()`.
`Bnew`	data.frame	Candidate model matrix. Contains all M existing columns of `B` plus two new columns (`Btem1`, `Btem2`) for the proposed child basis functions.

3. Step-by-Step Algorithm Walkthrough

3.1 Step 1: Initialization

The function begins by extracting the sample size N, the number of predictors n, and `Mmax` from the control object. It initializes `B` via `init_B(N, Mmax)`, which creates a data frame with column `B0` set to all 1s and columns `B1, ..., BMmax` set to 0. The list `Bfuncs` is initialized as an empty list of length $M_{\max} + 1$, with element 1 explicitly set to `NULL`.

```
N    <- length(y)
n    <- ncol(x)
Mmax <- control$Mmax

B      <- init_B(N, Mmax)                        # N x (Mmax+1)
Bfuncs <- vector("list", length = Mmax + 1)       # all NULL
Bfuncs[[1]] <- NULL                               # intercept
```

3.2 Step 2: Outer Loop — Iterating Over Pairs

Because we add basis functions in mirror-image *pairs*, the outer loop runs $M_{\max}/2$ times. The loop variable `i` ranges from 1 to $M_{\max}/2$, and the current number of non-constant basis functions is $M = 2i - 1$. This means M takes values 1, 3, 5, ... up to $M_{\max} - 1$. At each iteration, two new basis functions (columns $M + 1$ and $M + 2$ of `B`) will be added.

```
for (i in 1:(Mmax / 2)) {
    M          <- 2 * i - 1
    lof_best   <- Inf
    split_best <- NULL
    ...
}
```

3.3 Step 3: Inner Loops — Searching for the Best Split

Three nested loops search over: (a) which existing basis function m to split (from 1 to M), (b) which predictor variable v to use (restricted to variables not already in B_m), and (c) which knot t to place (from the candidate split points where $B_m > 0$).

Variable eligibility is determined by checking `Bfuncs[[m]]`: if `Bfuncs[[m]]` is `NULL` (the intercept), all variables are eligible (`svars = 1:n`). Otherwise, variables whose indices appear in the `v` column of `Bfuncs[[m]]` are excluded via `setdiff(1:n, Bfuncs[[m]][, "v"])`. This is the key mechanism that prevents the same variable from being used more than once in a single basis function, limiting the order of interactions.

For each candidate triple (m, v, t), a candidate model matrix `Bnew` is constructed containing all M existing basis function columns **plus** two new columns:

```
Bnew <- data.frame(B[, 1:M],
                   Btem1 = B[, m] * h(x[, v],  1, t),
                   Btem2 = B[, m] * h(x[, v], -1, t))
```

```
gdat <- data.frame(y = y, Bnew)
lof  <- LOF(y ~ ., gdat, control)
```

Note that during the search, the LOF formula uses `y ~ .` (with the implicit intercept from `lm()`) rather than `y ~ . - 1`. If the GCV is lower than `lof_best`, the candidate is stored in `split_best`.

3.4 Step 4: Updating B and Bfuncs

After the best split (m^*, v^*, t^*) is found, the function extracts the values using `unname()` to avoid named-vector issues with column names:

B matrix: Column $M + 1$ receives the **negative** hinge child, and column $M + 2$ receives the **positive** hinge child:

```
mstar <- unname(split_best["m"])
vstar <- unname(split_best["v"])
tstar <- unname(split_best["t"])

B[, M + 1] <- B[, mstar] * h(x[, vstar], -1, tstar)
B[, M + 2] <- B[, mstar] * h(x[, vstar],  1, tstar)
```

The parent column `B[, mstar]` is left unchanged.

Bfuncs list: The new entry `Bfuncs[[M+1]]` copies the parent's definition and appends a row with $s = -1$. Entry `Bfuncs[[M+2]]` copies the parent and appends $s = +1$:

```
Bfuncs[[M + 1]] <- rbind(Bfuncs[[mstar]], c(s = -1, v = vstar, t = tstar))
Bfuncs[[M + 2]] <- rbind(Bfuncs[[mstar]], c(s =  1, v = vstar, t = tstar))
```

After the loop completes, column names of `B` are reset to `B0, B1, ..., BMmax` and the function returns `list(y = y, B = as.data.frame(B), Bfuncs = Bfuncs)`.

4. Worked Example: Step-by-Step Trace

Consider the course test data with $n = 10$ observations, `x1` and `x2` as predictors, and $M_{\max} = 2$.

4.0.1 Iteration $i = 1$ ($M = 1$)

We search over all basis functions (only $B_0 = 1$ at this point), all variables (`x1`, `x2`), and all candidate knots. The algorithm evaluates every possible hinge-function pair and selects the one minimizing GCV. Suppose the best split is $m^* = 1, v^* = 1, t^* = -0.1389$. Then:

$$B_2(\mathbf{x}) = B_0(\mathbf{x}) \cdot h(x_1, -1, -0.1389) = \max(0,\ 0.1389 - x_1)$$
$$B_3(\mathbf{x}) = B_0(\mathbf{x}) \cdot h(x_1, +1, -0.1389) = \max(0,\ x_1 + 0.1389)$$

With $M_{\max} = 2$ (one iteration), the forward pass stops here.

4.0.2 Result

The output `B` data frame has columns `B0` (all 1s), `B1` (negative hinge on `x1`), and `B2` (positive hinge on `x1`). The `Bfuncs` list records:

- `Bfuncs[[1]]` = `NULL` (intercept)
- `Bfuncs[[2]]` = `matrix(c(-1, 1, -0.1389), ncol=3)` (negative hinge)
- `Bfuncs[[3]]` = `matrix(c(+1, 1, -0.1389), ncol=3)` (positive hinge)

5. The Hinge Function h()

The hinge function `h(x, s, t)` is implemented as:

```
h <- function(x, s, t) {
    pmax(0, s * (x - t))
}
```

When $s = +1$, it returns $\max(0, x - t)$ (a "hockey stick" rising above t). When $s = -1$, it returns $\max(0, t - x)$ (a mirror image falling below t). The use of `pmax()` ensures vectorized operation over all observations simultaneously.

6. Support Functions

6.1 init_B(N, Mmax)

Creates the initial $N \times (M_{\max} + 1)$ basis-function data frame. Column 1 (`B0`) is filled with 1s for the intercept; all remaining columns are initialized to **0** (not `NA`, to avoid issues with `lm()` during the search phase). Columns are named `B0, B1, ..., BMmax`.

```
init_B <- function(N, Mmax) {
    B        <- as.data.frame(matrix(0, nrow = N, ncol = Mmax + 1))
    B[, 1]   <- 1
    names(B) <- c("B0", paste0("B", 1:Mmax))
    B
}
```

6.2 split_points(xv, Bm)

Returns the vector of candidate knot locations for variable `xv` given parent basis function `Bm`. Specifically, it returns the sorted unique values of `xv` where $B_m > 0$, excluding the largest value (to ensure both children have non-empty support).

```
split_points <- function(xv, Bm) {
    out <- sort(unique(xv[Bm > 0]))
    out[-length(out)]
}
```

6.3 LOF(form, data, control)

Computes the GCV criterion. It fits `lm(form, data)`, extracts RSS, determines M from the number of coefficients minus one, computes $\widetilde{C}(M) = \text{sum}(\text{hatvalues}(\text{ff})) + d \cdot M$, and returns $N \cdot \text{RSS}/(N - \widetilde{C}(M))^2$.

```
LOF <- function(form, data, control) {
    ff     <- lm(form, data)
    RSS    <- sum(residuals(ff)^2)
    N      <- nrow(data)
    M      <- length(coef(ff)) - 1
    Ctilde <- sum(hatvalues(ff)) + control$d * M
    RSS * N / (N - Ctilde)^2
}
```

Part III

Backward Elimination Algorithm

1. Overview

Algorithm 3 of Friedman (1991) describes the backward pruning (elimination) phase of MARS. While the forward pass deliberately overfits by adding up to $M_{\max}$ basis functions, the backward pass corrects this by iteratively removing the least important basis function at each step, using GCV as the selection criterion.

2. Notation and Symbols

Symbol	Meaning
M	Current number of non-constant basis functions in the model.
K^* (`J_star`)	The set of column indices currently under consideration. At the start, $K^* = \{1, 2, \ldots, M_{\max} + 1\}$ (all columns of `B`).
$K_m = K^* \setminus \{m\}$	The subset obtained by removing the m-th element from K^*.
$\mathrm{LOF}(K)$	The GCV lack-of-fit evaluated on the model using only the basis-function columns in set K, with formula `y ~ . - 1`.
J^* (`J_best`)	The overall best subset found across all sizes, i.e., the one achieving the global minimum GCV.
$b(M)$	At each reduction step, the index whose removal yields the lowest GCV among all candidates.

3. Algorithm Description

3.1 Step 1: Initialization

Let $J^* = K^* = \{1, 2, \ldots, \mathrm{Ncols}\}$ be the full set of column indices from the forward pass. Compute $\mathrm{LOF}(J^*)$ using the formula `y ~ . - 1`, and set lof_star $= \mathrm{LOF}(J^*)$ and J_best $= J^*$.

```
J_star   <- 1:Ncols
lof_star <- LOF(y ~ . - 1,
                data = cbind(y = y, B[, J_star, drop = FALSE]),
                control)
J_best   <- J_star
```

3.2 Step 2: Iterative Removal

The outer loop runs from `Ncols` down to 2. At each step, every non-intercept element of the current `J_star` is tried for removal:

(a) For each position m from 2 to M (skipping position 1, which is the intercept `B0`), compute the GCV of the reduced model `LOF(y ~ . - 1, K)` where $K = \text{J_star}[-m]$.

(b) Track which removal yields the lowest GCV (`b_best`) and save the corresponding subset as `K_best`.

(c) Update K^*: set `J_star = K_best` (the subset with the best single removal).

(d) Separately, if *any* evaluated GCV is lower than `lof_star`, update the global best: `J_best = K` and `lof_star = lof`.

```
for (M in Ncols:2) {
    b_best <- Inf
    K_best <- NULL
    for (m in 2:M) {
        K   <- J_star[-m]
        dat <- cbind(y = y, B[, K, drop = FALSE])
        lof <- LOF(y ~ . - 1, data = dat, control)
        if (lof < b_best) { b_best <- lof; K_best <- K }
        if (lof < lof_star) { lof_star <- lof; J_best <- K }
    }
    J_star <- K_best
}
```

3.3 Step 3: Final Model

After all iterations, `J_best` contains the column indices of the basis functions in the best model. The backward pass returns the corresponding columns of `B` and the matching entries of `Bfuncs`:

```
list(y      = y,
     B      = B[, J_best, drop = FALSE],
     Bfuncs = Bfuncs[J_best])
```

4. Why Backward Pruning Is Necessary

The forward pass is greedy: each pair of basis functions is chosen to maximize the immediate reduction in GCV, without considering whether those terms will remain useful once later terms are added. This often produces a model with redundant or collinear basis functions. The backward pass addresses this by considering the joint contribution of each term in the context of the full model, producing a sparser model that typically generalizes better to unseen data.

The two-pass approach is analogous to forward-backward stepwise regression in classical statistics, but uses the MARS-specific GCV criterion rather than AIC or BIC, and operates on the specialized basis-function space rather than the original predictors.

5. Illustrative Example

Suppose the forward pass with $M_{\max} = 4$ produces basis functions $\{B_0, B_1, B_2, B_3, B_4\}$ (column indices 1–5) with GCV = 2.45. The backward pass proceeds as follows:

Step	Current K^*	Term Removed	GCV	Best So Far?
0	$\{B_0, B_1, B_2, B_3, B_4\}$	—	2.45	$J^* = \{1, 2, 3, 4, 5\}$
1	$\{B_0, B_1, B_2, B_4\}$	B_3	2.31	J^* updated
2	$\{B_0, B_1, B_4\}$	B_2	2.18	J^* updated
3	$\{B_0, B_4\}$	B_1	2.52	No improvement

In this example, the best model is $\{B_0, B_1, B_4\}$ with GCV = 2.18. Removing B_3 and B_2 improved the model by eliminating overfitting, but removing B_1 further would degrade performance. The final model retains only the intercept and two basis functions.

6. Comparison to Forward Pass Only

Without backward pruning, the forward pass would deliver a model with all $M_{\max}$ basis functions, many of which may capture noise rather than signal. Empirically, the backward-pruned model almost always achieves equal or better predictive accuracy on held-out data, while using fewer parameters. This parsimony also improves interpretability — the analyst can examine the surviving basis functions to understand which variables and interaction terms are truly important.